# Database ERD: Black Friday Platform

The PostgreSQL 16 database `fridayblack` (with pgvector), as read from the live database after Alembic migration `0002`.

---

## 1. Overview

The database has **10 tables in 4 groups**:

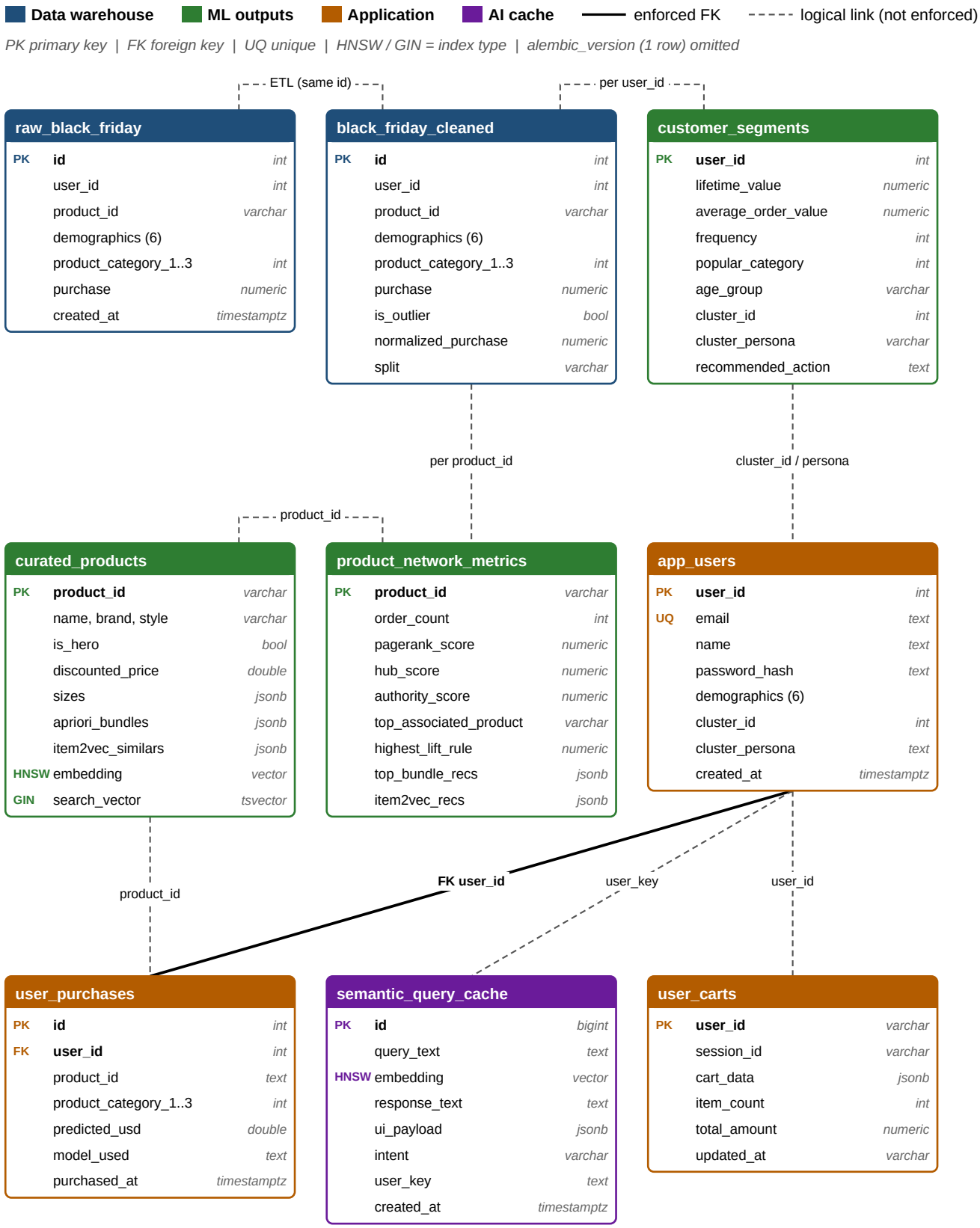
| Group | Tables | Written by | Rows (live) |
|---|---|---|---|
| **Data warehouse** | `raw_black_friday`, `black_friday_cleaned` | `make ingest`, `make preprocess` | 550,069 each |
| **ML outputs** | `customer_segments`, `product_network_metrics`, `curated_products` | `make segmentation`, `make basket`, `seed_curated` | 5,891 / 133 / 50 |
| **Application** | `app_users`, `user_purchases`, `user_carts` | The FastAPI backend | 1,632 / 117 / 0 |
| **AI cache and system** | `semantic_query_cache`, `alembic_version` | The AI assistant, Alembic | 0 / 1 |

**Enforced vs logical relationships.** Only **one** relationship is enforced by a real foreign key (`user_purchases.user_id` references `app_users.user_id`). The others are **logical**: the columns match, but the database does not check them. That is normal for an analytics warehouse loaded in bulk, where foreign keys would slow down the 550k-row loads and block truncate-and-reload.

---

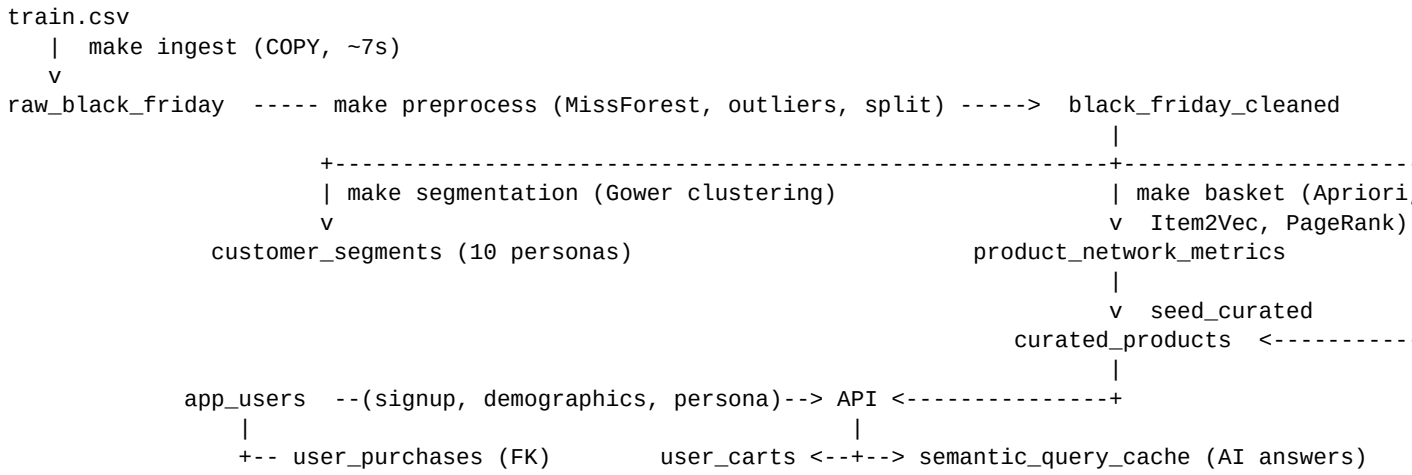
## 2. Entity relationship diagram

Solid line = enforced foreign key, dashed line = logical relationship.


Data warehouse
ML outputs
Application
AI cache
enforced FK
logical link (not enforced)
PK primary key | FK foreign key | UQ unique | HNSW / GIN = index type | alembic_version (1 row) omitted
ETL (same id)
per user_id
raw_black_friday
PK id int
user_id int
product_id varchar
demographics (6)
product_category_1..3 int
purchase numeric
created_at timestamptz
black_friday_cleaned
PK id int
user_id int
product_id varchar
demographics (6)
product_category_1..3 int
purchase numeric
is_outlier bool
normalized_purchase numeric
split varchar
customer_segments
PK user_id int
lifetime_value numeric
average_order_value numeric
frequency int
popular_category int
age_group varchar
cluster_id int
cluster_persona varchar
recommended_action text
per product_id
cluster_id / persona
product_id
curated_products
PK product_id varchar
name, brand, style varchar
is_hero bool
discounted_price double
sizes jsonb
apriori_bundles jsonb
item2vec_similars jsonb
HNSW embedding vector
GIN search_vector tsvector
product_network_metrics
PK product_id varchar
order_count int
pagerank_score numeric
hub_score numeric
authority_score numeric
top_associated_product varchar
highest_lift_rule numeric
top_bundle_recs jsonb
item2vec_recs jsonb
app_users
PK user_id int
UQ email text
name text
password_hash text
demographics (6)
cluster_id int
cluster_persona text
created_at timestamptz
product_id
FK user_id
user_key
user_id
user_purchases
PK id int
FK user_id int
product_id text
product_category_1..3 int
predicted_usd double
model_used text
purchased_at timestamptz
semantic_query_cache
PK id bigint
query_text text
HNSW embedding vector
response_text text
ui_payload jsonb
intent varchar
user_key text
created_at timestamptz
user_carts
PK user_id varchar
session_id varchar
cart_data jsonb
item_count int
total_amount numeric
updated_at varchar

> The diagram above renders on GitHub and in VS Code with a Mermaid extension. The PDF version of this document has the same diagram drawn as a picture.

## 3. How the data flows




## 4. Tables in detail

### *4.1 raw_black_friday (data warehouse)*

The original transactions, exactly as in `train.csv`. **Never modified after loading.**

| Column | Type | Null | Notes |
|---|---|---|---|
| `id` | integer | no | **PK** |
| `user_id` | integer | no | Dataset customer id (indexed) |
| `product_id` | varchar(32) | no | e.g. `P00025442` (indexed) |
| `gender`, `age`, `occupation`, `city_category`, `stay_in_current_city_years`, `marital_status` | varchar / int | no | Customer demographics |
| `product_category_1` | integer | no | Indexed |
| `product_category_2`, `product_category_3` | integer | **yes** | Missing in the raw data |
| `purchase` | numeric(12,2) | no | Amount in INR (the regression target) |
| `created_at` | timestamptz | yes | Load time |

### *4.2 black_friday_cleaned (data warehouse)*

Same rows as raw, after cleaning: categories 2 and 3 **imputed** by MissForest, outliers flagged, target normalised, and a train/test split assigned.

| Column | Type | Null | Notes |
|---|---|---|---|
| `id` | integer | no | **PK**, same id as the raw row |

| Column | Type | Null | Notes |
|---|---|---|---|
| `user_id`, `product_id` | integer / varchar(32) | no | Indexed |
| demographics + `product_category_1..3` | | no | Categories 2 and 3 are now **never null** (imputed) |
| `purchase` | numeric(12,2) | no | |
| `is_outlier` | boolean | no | Indexed; outlier rows are excluded from drift batches |
| `normalized_purchase` | numeric(8,6) | no | The model's training target |
| `split` | varchar(16) | no | `train` or `test` (indexed); fixed before any fitting, so no leakage |

### *4.3 customer_segments (ML output)*

One row per dataset customer, with RFM-style features and their cluster.

| Column | Type | Notes |
|---|---|---|
| `user_id` | integer | **PK** (dataset customer id) |
| `lifetime_value`, `average_order_value` | numeric | Spending features (LTV indexed) |
| `frequency` | integer | Distinct products bought (PIPE-13 explains why it isn't a purchase count) |
| `purchase_amount_variability`, `popular_category` | numeric / int | |
| `gender`, `marital_status`, `age_group`, `age_binned` | varchar | |
| `cluster_id` | integer | 0–9 (indexed) |
| `cluster_persona`, `recommended_action` | varchar / text | One of the **10 personas** and its marketing action |

### *4.4 product_network_metrics (ML output)*

Graph and recommendation scores per product, from Apriori rules and the product co-purchase network.

| Column | Type | Notes |
|---|---|---|
| `product_id` | varchar(32) | **PK** |
| `order_count` | integer | |
| `pagerank_score`, `hub_score`, `authority_score` | numeric(8,5) | Network centrality (each indexed) |
| `top_associated_product`, `highest_lift_rule` | varchar / numeric | The best Apriori rule |
| `top_bundle_recommendations`, `item2vec_recommendations` | jsonb | Lists of recommended products |

### *4.5 curated_products (store catalog)*

The 50 products shown in the store, enriched with names, images, prices and recommendations.

| Column | Type | Notes |
|---|---|---|
| `product_id` | varchar(32) | **PK** |
| `name`, `title`, `tagline`, `description`, `brand`, `style`, `season`, `badge` | varchar / text | Display text |
| `category_name`, `category`, `gender` | varchar | Used by the sidebar filters |
| `is_hero` | boolean | The featured product on the right of the page |
| `original_price`, `discounted_price` | double | Prices shown in the UI |
| `rating`, `review_count`, `order_count` | double / int | |
| `sizes`, `apriori_bundles`, `item2vec_similars` | jsonb | Sizes and "bought together" / "similar" lists |
| `embedding` | vector | **pgvector** embedding for semantic search (**HNSW index**) |
| `search_vector` | tsvector | Full-text search (GIN index) |

### *4.6 app_users (application)*

Registered store users. Separate from the dataset customers in the warehouse.

| Column | Type | Notes |
|---|---|---|
| `user_id` | integer | **PK** |
| `email` | text | **UNIQUE** (indexed); stored lower-case |
| `name`, `password_hash` | text | bcrypt hash, never the password itself |
| `gender`, `age`, `city_category`, `occupation`, `marital_status`, `stay_in_current_city_years` | | Demographics, used as model inputs for personal price predictions |
| `cluster_id`, `cluster_persona`, `recommended_action` | | The persona assigned at signup |
| `created_at` | timestamptz | |

### *4.7 user_purchases (application)*

| Column | Type | Notes |
|---|---|---|
| `id` | integer | **PK** |
| `user_id` | integer | **FK to** `app_users.user_id`, not null (indexed) |
| `product_id` | text | The product bought |
| `product_category_1..3` | integer | |

| Column | Type | Notes |
|---|---|---|
| `predicted_usd` | double | The ONNX price prediction at purchase time |
| `model_used` | text | Which model made the prediction |
| `purchased_at` | timestamptz | |

### *4.8 user_carts (application)*

Durable cart storage ("cold tier"; Redis is the fast tier).

| Column | Type | Notes |
|---|---|---|
| `user_id` | varchar(64) | **PK**: JWT subject, or a `guest_<hex>` id |
| `session_id` | varchar(64) | Browser session |
| `cart_data` | jsonb | Cart items |
| `item_count`, `total_amount` | int / numeric | |
| `updated_at` | varchar(64) | Stored as text (see section 6) |

### *4.9 semantic_query_cache (AI)*

Caches the assistant's answers by meaning, so similar questions reuse an answer.

| Column | Type | Notes |
|---|---|---|
| `id` | bigint | **PK** |
| `query_text`, `response_text` | text | The question and the answer |
| `embedding` | vector | The question's embedding (**HNSW index** for nearest-neighbour search) |
| `ui_payload` | jsonb | Product cards shown with the answer |
| `intent` | varchar(64) | The router's intent label |
| `user_key` | text | Owner of a personalised answer, so it isn't served to someone else (CORE-15) |
| `created_at` | timestamptz | Indexed together with `user_key` |

### *4.10 alembic_version (system)*

One row with the current migration version (`0002`). Managed by Alembic only; never edit it by hand.

---

## 5. Special column types

| Type | What it is | Used for |
|---|---|---|
| `jsonb` | Binary JSON: flexible nested data that can still be queried and indexed | Bundles, recommendations, cart contents, UI payloads |
| `vector` | **pgvector**: a list of numbers (an embedding) | Semantic search and the semantic cache |

| Type | What it is | Used for |
|---|---|---|
| HNSW index | A fast **approximate nearest-neighbour** index for vectors | "Find the most similar product or question" |
| `tsvector` + GIN index | PostgreSQL full-text search | Keyword search in the catalog |
| `numeric(p,s)` | Exact decimal (p digits, s after the point) | Money and scores; no floating-point rounding errors |
| `timestamptz` | Timestamp with time zone | Audit times |

---

## 6. Observations

These are facts noticed while drawing this ERD. They are not covered by the review findings so far:

1. **Prices in** `curated_products` **and** `user_purchases.predicted_usd` **use** `double precision`, while the warehouse uses exact `numeric`. Money is usually stored as `numeric`.
2. `user_carts.updated_at` **is a** `varchar`, not a timestamp, so it cannot be sorted or filtered as a time.
3. **Name mismatches between the app and the warehouse:** `app_users.gender` / `city_category` are `varchar(1)`, `user_purchases.product_id` is `text`, while the warehouse uses `varchar(8)` / `varchar(4)` / `varchar(32)`.
4. **Only one foreign key.** Logical links (for example `user_purchases.product_id` to `curated_products`) are not enforced. That's fine for the warehouse; for the application tables, a foreign key could be considered.
5. **Two kinds of "user":** `customer_segments.user_id` is a **dataset customer**, while `app_users.user_id` is a **store account**. They're different id spaces that share a column name.